
Documentação de um Produto de Software

Versão 3.0

Autor: Squad 78

InvoiSys

ÍNDICE DETALHADO

PREFÁCIO	4
1. INTRODUÇÃO AO DOCUMENTO	6
1.1. TEMA	6
1.2. OBJETIVO DO PROJETO	6
1.3. DELIMITAÇÃO DO PROBLEMA	6
1.4. JUSTIFICATIVA DA ESCOLHA DO TEMA	6
1.5. MÉTODO DE TRABALHO	6
1.6. ORGANIZAÇÃO DO TRABALHO	7
1.7. GLOSSÁRIO	7
2. DESCRIÇÃO GERAL DO SISTEMA	8
2.1. DESCRIÇÃO DO PROBLEMA	8
2.2. PRINCIPAIS ENVOLVIDOS E SUAS CARACTERÍSTICAS	8
2.3. REGRAS DE NEGÓCIO	8
3. REQUISITOS DO SISTEMA	9
3.1. REQUISITOS FUNCIONAIS	9
3.2. REQUISITOS NÃO-FUNCIONAIS	10
3.3. PROTÓTIPO	10
3.4. MÉTRICAS E CRONOGRAMA	11
4. ANÁLISE E DESIGN	12
4.1. ARQUITETURA DO SISTEMA	12
4.2. MODELO DO DOMÍNIO	12
4.3. DIAGRAMAS DE INTERAÇÃO	13
4.4. DIAGRAMA DE CLASSES	14
4.5. DIAGRAMA DE ATIVIDADES	14
4.6. DIAGRAMA DE ESTADOS	14
4.7. DIAGRAMA DE COMPONENTES	15
4.8. MODELO DE DADOS	16
4.8.1. <i>Modelo Lógico da Base de Dados</i>	16
4.8.2. <i>Criação Física do Modelo de Dados</i>	16
4.8.3. <i>Dicionário de Dados</i>	16
4.9. AMBIENTE DE DESENVOLVIMENTO	16
4.10. SISTEMAS E COMPONENTES EXTERNOS UTILIZADOS	16
5. IMPLEMENTAÇÃO	17
6. TESTES	18
6.1. PLANO DE TESTES	18
6.2. EXECUÇÃO DO PLANO DE TESTES	18
7. IMPLANTAÇÃO	19
7.1. DIAGRAMA DE IMPLANTAÇÃO	19
7.2. MANUAL DE IMPLANTAÇÃO	19
8. MANUAL DO USUÁRIO	20

9. CONCLUSÕES E CONSIDERAÇÕES FINAIS	21
BIBLIOGRAFIA	22

Prefácio

O objetivo deste documento é fornecer um roteiro para o desenvolvimento de sistemas de software utilizando os princípios da engenharia de software orientada a objetos com notação UML (*Unified Modeling Language*). É destinado a todos os alunos da Universidade São Judas Tadeu dos cursos de Ciência da Computação, Sistemas de Informação e Processamento de Dados, apoiando as disciplinas de Metodologia de Desenvolvimento de Sistemas, Engenharia de Software I, Engenharia de Software II, entre outras, além do Trabalho de Graduação (TG).

Esta é a versão 3.0 do documento, totalmente revisada para utilizar a notação UML e modelos do RUP (*Rational Unified Process* – Processo Unificado *Rational*). Neste documento são citados alguns modelos do RUP que podem ser utilizados e consultados na ferramenta *Rational Unified Process* (que faz parte da ferramenta *Rational Suite Enterprise*) ou o site da IBM. A escolha da orientação a objetos é devido à tendência de mercado, mas nada impede que o roteiro seja seguido no caso de opção pela Modelagem Estruturada (ver item Comentários sobre a Documentação).

No final deste documento há um glossário, os termos que constam no glossário são representados no documento pela letra ^G em azul.

Sugestões e Comentários podem ser enviados para prof.anapaula@usjt.br.

Modelo da Documentação

Esta é a parte mais importante do texto pois apresenta um roteiro de documentação orientado a objetos de sistemas de software utilizando notação UML, desde a fase inicial do projeto de software até a sua implantação.

Para a criação dos diagramas aconselha-se a utilização de alguma ferramenta CASE [G](#).

1. Introdução ao Documento

Este documento apresenta a análise, os requisitos e a modelagem da Plataforma Inteligente para Geração Automatizada de Release Notes da InvoiSys. O sistema tem como finalidade apoiar a elaboração de comunicados de Release a partir das informações registradas no Jira, utilizando Inteligência Artificial para classificação, consolidação e geração de conteúdo, com revisão e aprovação humana antes da publicação.

A documentação foi estruturada até a seção 4.9 do modelo de documentação de produto de software, contemplando requisitos, protótipos, arquitetura, modelos de domínio, interação, classes, atividades, estados, componentes, dados e ambiente de desenvolvimento.

1.1. Tema

Plataforma Inteligente para Geração Automatizada de Release Notes utilizando Inteligência Artificial e integração com Jira.

1.2. Objetivo do Projeto

Desenvolver um protótipo funcional que utilize Inteligência Artificial integrada à API do Jira para apoiar a elaboração das Release Notes da InvoiSys, identificando alterações relevantes, consolidando informações semelhantes e gerando um comunicado preliminar para revisão, edição e aprovação do usuário antes da publicação.

1.3. Delimitação do Problema

O projeto está delimitado ao desenvolvimento de um MVP capaz de obter dados de uma Release, preferencialmente por integração com o Jira, analisar histórias e subtarefas relacionadas, identificar informações relevantes ao cliente, eliminar duplicidades e gerar uma versão preliminar do comunicado contendo título, resumo e texto. O conteúdo gerado permanece sujeito à revisão humana, não abrangendo publicação automática sem aprovação.

1.4. Justificativa da Escolha do Tema

A escolha do tema está relacionada à necessidade de reduzir o esforço manual empregado na elaboração de Release Notes. A solução busca organizar informações técnicas, reduzir duplicidades e padronizar a comunicação das alterações do produto, mantendo a participação do usuário responsável pela revisão e aprovação do conteúdo.

1.5. Método de Trabalho

O desenvolvimento segue a metodologia ágil Scrum, organizado em Sprints incrementais. O trabalho é dividido entre levantamento e validação de requisitos, interface, back-end, Inteligência Artificial, integração com Jira, banco de dados, testes e documentação. A cada ciclo são avaliados os resultados produzidos e os ajustes necessários para a evolução do MVP.

1.6. Organização do Trabalho

O documento está organizado em etapas progressivas. Inicialmente são apresentados o tema, objetivos, delimitação, justificativa, método e glossário. Em seguida são descritos o problema, os envolvidos e as regras de negócio. O capítulo de requisitos apresenta as funcionalidades, requisitos não funcionais, protótipos e planejamento. Por fim, a seção de análise e design detalha a arquitetura, os modelos UML e o banco de dados até o ambiente de desenvolvimento.

1.7. Glossário

Termo	Definição
Release	Versão específica do produto que reúne histórias, correções, melhorias e novas funcionalidades.
Release Note	Comunicado que apresenta as alterações realizadas em determinada versão do produto.
Story	História que descreve uma necessidade, funcionalidade, melhoria ou alteração do sistema.
Jira	Sistema utilizado para gerenciamento de projetos, histórias, tarefas e Releases.
Subtarefa	Atividade vinculada a uma história principal.
IA Generativa	Tecnologia de Inteligência Artificial capaz de gerar conteúdos a partir de informações fornecidas.
LLM	Large Language Model, modelo de linguagem utilizado para interpretação e geração de textos.

MVP	Minimum Viable Product, versão mínima do sistema capaz de demonstrar a viabilidade da solução.
Human-in-the-loop	Abordagem em que uma pessoa participa da validação ou tomada de decisão junto ao sistema de IA.
Embedding	Representação numérica de um texto utilizada para comparar semanticamente diferentes conteúdos.
API	Interface que permite a comunicação entre diferentes sistemas.
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018).
Log de Auditoria	Registro das ações realizadas pelos usuários dentro do sistema para rastreabilidade e controle.
Rastreabilidade	Capacidade de relacionar informações geradas no comunicado às histórias que serviram como fonte.

2. Descrição Geral do Sistema

2.1. Descrição do Problema

A InvoiSys atua com soluções SaaS para gerenciamento de Documentos Fiscais Eletrônicos. Durante o desenvolvimento do produto, são registradas no Jira histórias relacionadas a novas funcionalidades, melhorias, correções e alterações técnicas. A cada Release, essas informações precisam ser analisadas para determinar quais mudanças devem ser comunicadas aos clientes.

O processo manual exige consultar histórias e subtarefas, interpretar informações técnicas, selecionar conteúdos relevantes, consolidar informações semelhantes, remover duplicidades e redigir o comunicado final em linguagem adequada ao cliente. Entre os principais impactos estão o esforço operacional elevado, a possibilidade de omissão de informações relevantes, a duplicação de conteúdo, a dificuldade de padronização e o tempo necessário para concluir cada comunicado.

A solução proposta utiliza Inteligência Artificial para auxiliar esse processo, mantendo a revisão humana como etapa obrigatória antes da aprovação.

2.2. Principais Envolvidos e suas Características

Envolvido	Características e responsabilidades
Cliente da InvoiSys	Destinatário das Release Notes; necessita de informações claras, objetivas, organizadas, relevantes e com pouco conteúdo técnico desnecessário.
Colaborador responsável pelas Release Notes	Principal usuário; seleciona ou consulta Releases, inicia o processamento, analisa o conteúdo gerado, realiza alterações e aprova a versão final quando autorizado.
Equipe de Desenvolvimento	Cria histórias e implementa alterações registradas no Jira; suas informações são fonte para o processamento.
Equipe de Produto / Negócio	Avalia se as alterações estão adequadamente representadas sob a perspectiva do negócio e do cliente.
Equipe de Suporte	Utiliza as informações das Releases para compreender alterações e apoiar clientes.
Equipe Comercial	Pode utilizar informações das Releases para compreender funcionalidades e melhorias relevantes comercialmente.
Administrador do Sistema	Configura a aplicação, integrações, usuários e permissões necessárias ao funcionamento da plataforma.

2.1.1. Usuários do Sistema

Ver tabela “Principais Envolvidos e suas Características” (item 2.2), que descreve o Cliente da InvoiSys como principal usuário externo do sistema.

2.1.2. Desenvolvedores do Sistema

Ver tabela “Principais Envolvidos e suas Características” (item 2.2), que descreve a Equipe de Desenvolvimento, a Equipe de Produto/Negócio e o Administrador do Sistema.

Regras de Negócio

ID	Regra	Descrição
RN01	Identificação da Release	O sistema deverá permitir identificar uma Release e obter suas histórias relacionadas.
RN02	Fonte das informações	As informações utilizadas na geração do comunicado deverão ser provenientes das histórias e subtarefas relacionadas à Release.
RN03	Release Note existente	Quando uma história possuir uma sub tarefa específica de Release Note, seu conteúdo deverá ser considerado prioritariamente durante a geração do comunicado.
RN04	Relevância para o cliente	O sistema deverá identificar alterações relevantes para o público externo e evitar apresentação desnecessária de informações exclusivamente técnicas ou internas.
RN05	Classificação	As alterações relevantes deverão ser classificadas, quando aplicável, em categorias como Novas Funcionalidades, Melhorias e Correções.
RN06	Duplicidade	Informações semanticamente semelhantes deverão ser agrupadas ou consolidadas para evitar repetição.
RN07	Linguagem	O comunicado destinado ao cliente deverá utilizar linguagem clara, objetiva e orientada ao negócio.
RN08	Não invenção de informações	A IA não deverá criar funcionalidades, comportamentos, números ou benefícios que não estejam presentes nas informações fornecidas como fonte.
RN09	Rastreabilidade	Cada informação relevante gerada deverá, sempre que possível, manter referência às histórias utilizadas como fonte.
RN10	Revisão humana	Uma Release Note gerada automaticamente deverá passar por revisão humana antes de ser considerada aprovada.
RN11	Edição	O usuário deverá poder editar o conteúdo gerado pela IA antes da aprovação.
RN12	Aprovação	Somente usuário autorizado poderá aprovar a versão final da Release Note.
RN13	Diferentes públicos	O sistema poderá gerar versões adaptadas para públicos como cliente, interno, comercial e suporte, conforme o escopo definido para o MVP.

RN14	Histórico	O sistema deverá preservar informações sobre versões geradas e revisadas, permitindo acompanhar alterações realizadas durante o processo.
RN15	Título e resumo executivo	O sistema deverá sugerir automaticamente título e resumo executivo, além do texto completo do comunicado.

3. Requisitos do Sistema

3.1. Requisitos Funcionais

Os requisitos funcionais descrevem as ações e funcionalidades disponibilizadas aos usuários. Cada requisito abaixo representa uma ação que pode ser executada na plataforma, sendo posteriormente passível de validação por testes.

ID	Requisito	Descrição
RF01	Selecionar Release	O usuário deverá poder selecionar uma Release disponível no Jira para iniciar o processo de geração da Release Note.
RF02	Consultar Release	O usuário deverá poder visualizar as informações da Release selecionada, incluindo versão, data, descrição e histórias relacionadas.
RF03	Filtrar Releases	O usuário deverá poder filtrar as Releases por critérios como versão, data e status.
RF04	Gerar Release Note	O usuário deverá poder iniciar o processamento de uma Release por meio da ação de geração da Release Note. O sistema deverá utilizar as informações disponíveis para gerar uma versão preliminar do comunicado.
RF05	Visualizar Release Note	O usuário deverá poder visualizar o título, resumo e conteúdo da Release Note gerada pelo sistema.
RF06	Editar Release Note	O usuário deverá poder editar o conteúdo da Release Note antes de sua aprovação.
RF07	Solicitar Nova Geração	O usuário deverá poder solicitar uma nova geração da Release Note, informando comentários ou orientações para ajuste do conteúdo.
RF08	Aprovar Release Note	Um usuário autorizado deverá poder aprovar a Release Note após sua revisão.
RF09	Consultar Histórico	O usuário autorizado deverá poder consultar o histórico de versões geradas e modificadas de uma Release Note.
RF10	Visualizar Versão	O usuário deverá poder selecionar uma versão do histórico para visualizar seu conteúdo.
RF11	Criar Usuário	O administrador deverá poder cadastrar novos usuários na plataforma.
RF12	Editar Usuário	O administrador deverá poder alterar os dados de um usuário cadastrado.

RF13	Desativar Usuário	O administrador deverá poder desativar um usuário existente.
RF14	Definir Permissão	O administrador deverá poder definir o nível de acesso de cada usuário.
RF15	Configurar Jira	O administrador deverá poder cadastrar ou alterar as configurações necessárias para integração com o Jira.
RF16	Consultar Logs	Um usuário autorizado deverá poder consultar os registros das ações realizadas na plataforma.

Os processos internos executados automaticamente, como consulta ao Jira, processamento das histórias pela IA, classificação, consolidação e eliminação de duplicidades, não são tratados como requisitos funcionais independentes. Eles fazem parte da execução das funcionalidades correspondentes e são complementados pelas regras de negócio e requisitos não funcionais.

3.2. Requisitos Não-Funcionais

ID	Requisito	Descrição
RNF01	Segurança	O sistema deverá proteger as informações obtidas do Jira e restringir o acesso às funcionalidades conforme as permissões do usuário.
RNF02	Privacidade e Conformidade (LGPD)	O sistema deverá tratar dados pessoais e de clientes em conformidade com a LGPD, incluindo controle de acesso aos dados armazenados.
RNF03	Desempenho	O sistema deverá processar uma Release em tempo adequado, considerando a quantidade de histórias e o tempo de resposta do serviço de IA.
RNF04	Disponibilidade	A aplicação deverá permanecer disponível durante o horário de utilização pelos usuários autorizados.
RNF05	Usabilidade	A interface deverá ser simples e intuitiva, permitindo que usuários sem conhecimento técnico em IA consigam gerar e revisar uma Release Note.
RNF06	Manutenibilidade	O sistema deverá possuir arquitetura modular, permitindo substituir ou evoluir os componentes de integração, IA, banco de dados e interface.
RNF07	Escalabilidade	A arquitetura deverá suportar aumento no volume de Releases processadas e de usuários sem reestruturação completa.
RNF08	Qualidade da Geração da IA	A qualidade das gerações deverá ser acompanhada por métricas que permitam avaliar a necessidade de reformulação dos comunicados antes da aprovação.
RNF09	Observabilidade	O sistema deverá registrar eventos relevantes, erros e etapas de processamento para facilitar o diagnóstico de problemas.

RNF10	Integração	A solução deverá possuir mecanismos adequados de comunicação com a API do Jira e com o serviço de IA escolhido.
RNF11	Testabilidade	Os principais componentes deverão poder ser submetidos a testes unitários, funcionais e integrados.

3.3. Protótipo

Os protótipos foram elaborados no Figma e representam a interface proposta para o MVP. As telas apresentadas nesta seção devem ser tratadas como artefatos de análise e design, não como prova de que todas as funcionalidades já estão implementadas no código. No estado atual do repositório, o frontend implementa o fluxo de autenticação com três telas (Login, recuperação de senha e confirmação), enquanto as telas de Dashboard, revisão, configurações, usuários e logs permanecem como protótipos/escopo de evolução.

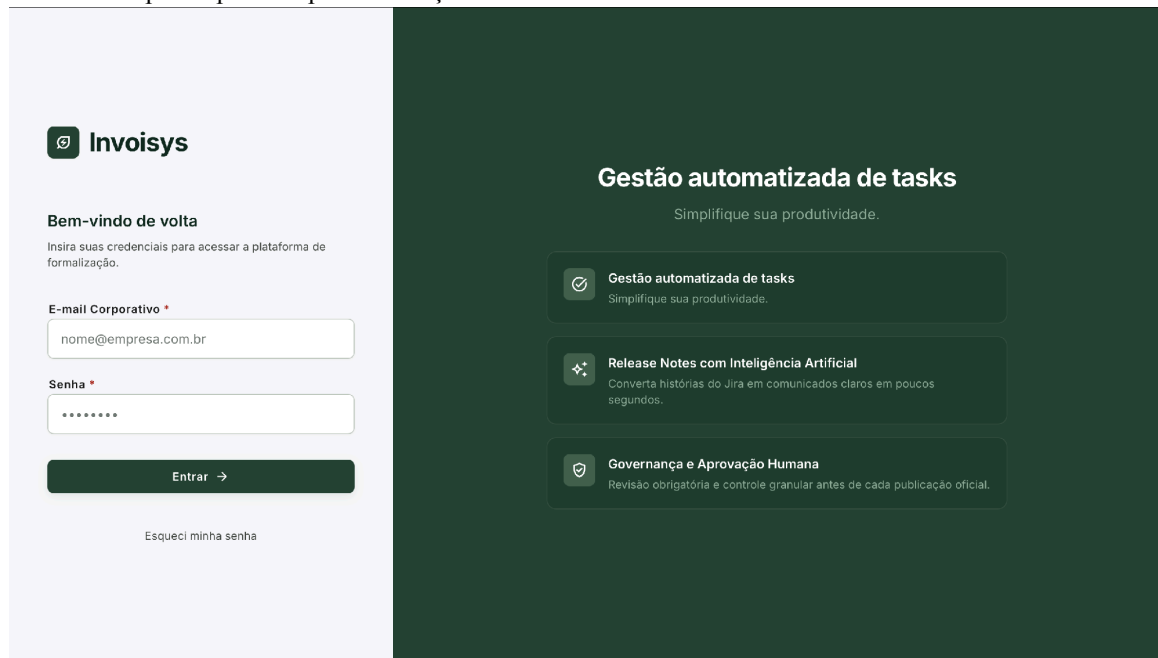


Figura — Tela de Login.

Permite a autenticação do usuário na plataforma.

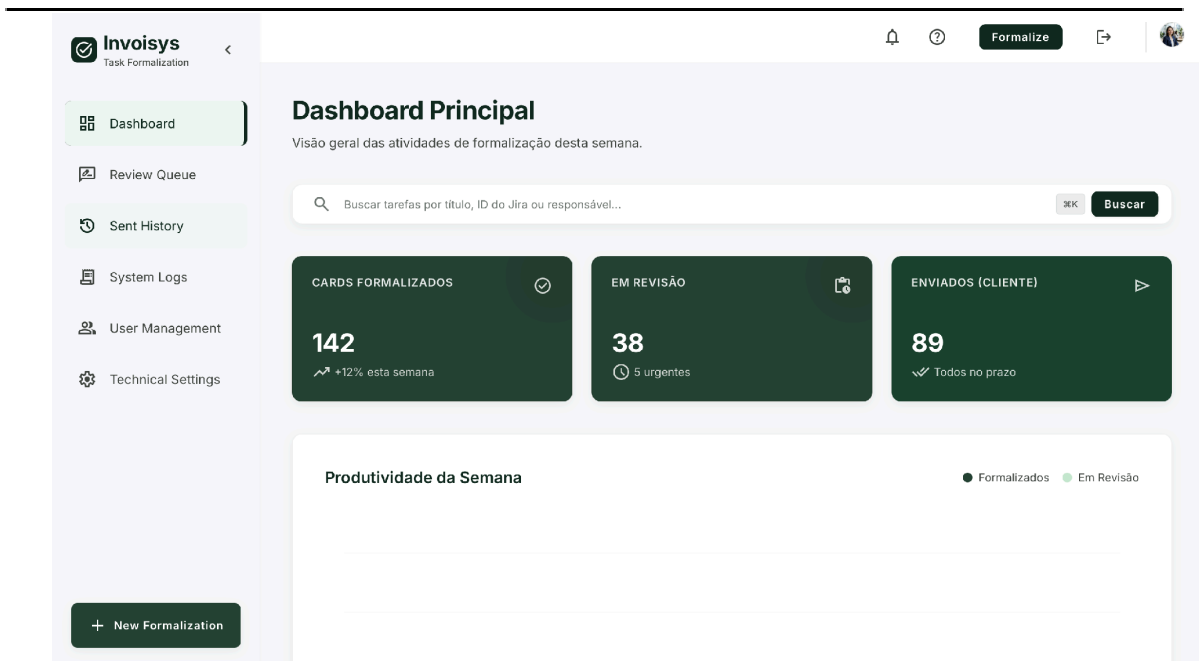


Figura — Dashboard Principal.

Apresenta indicadores de formalizações, itens em revisão e itens enviados ao cliente, além do acesso às principais áreas.

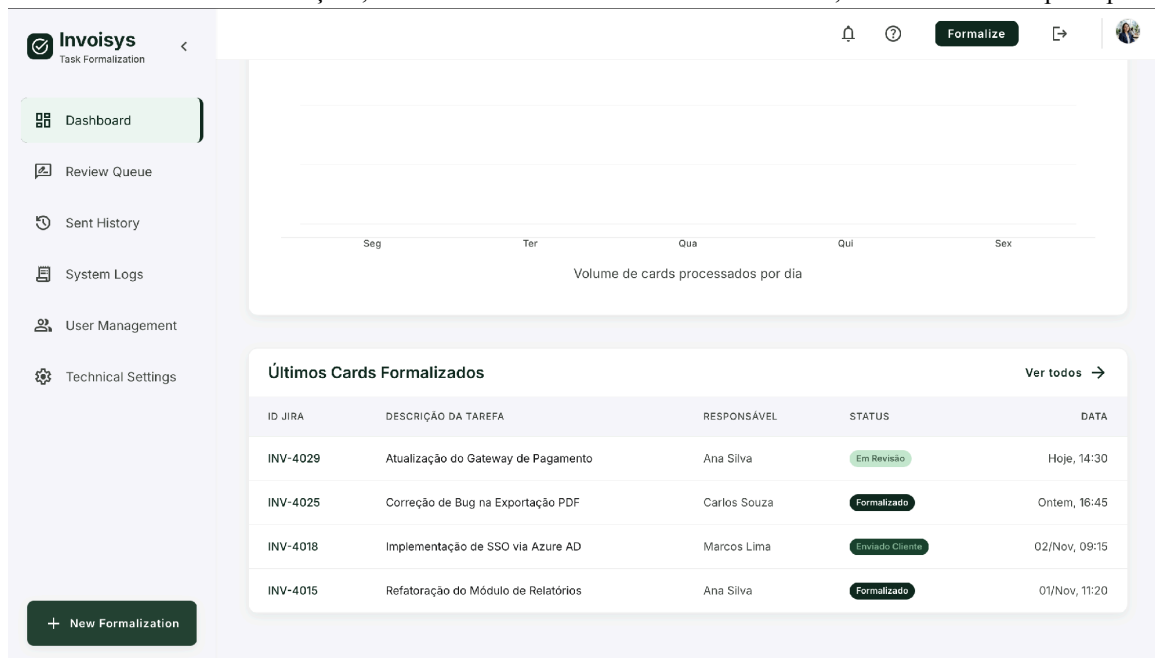


Figura — Dashboard com histórico.

Apresenta o volume de cards formalizados e a listagem das últimas formalizações.

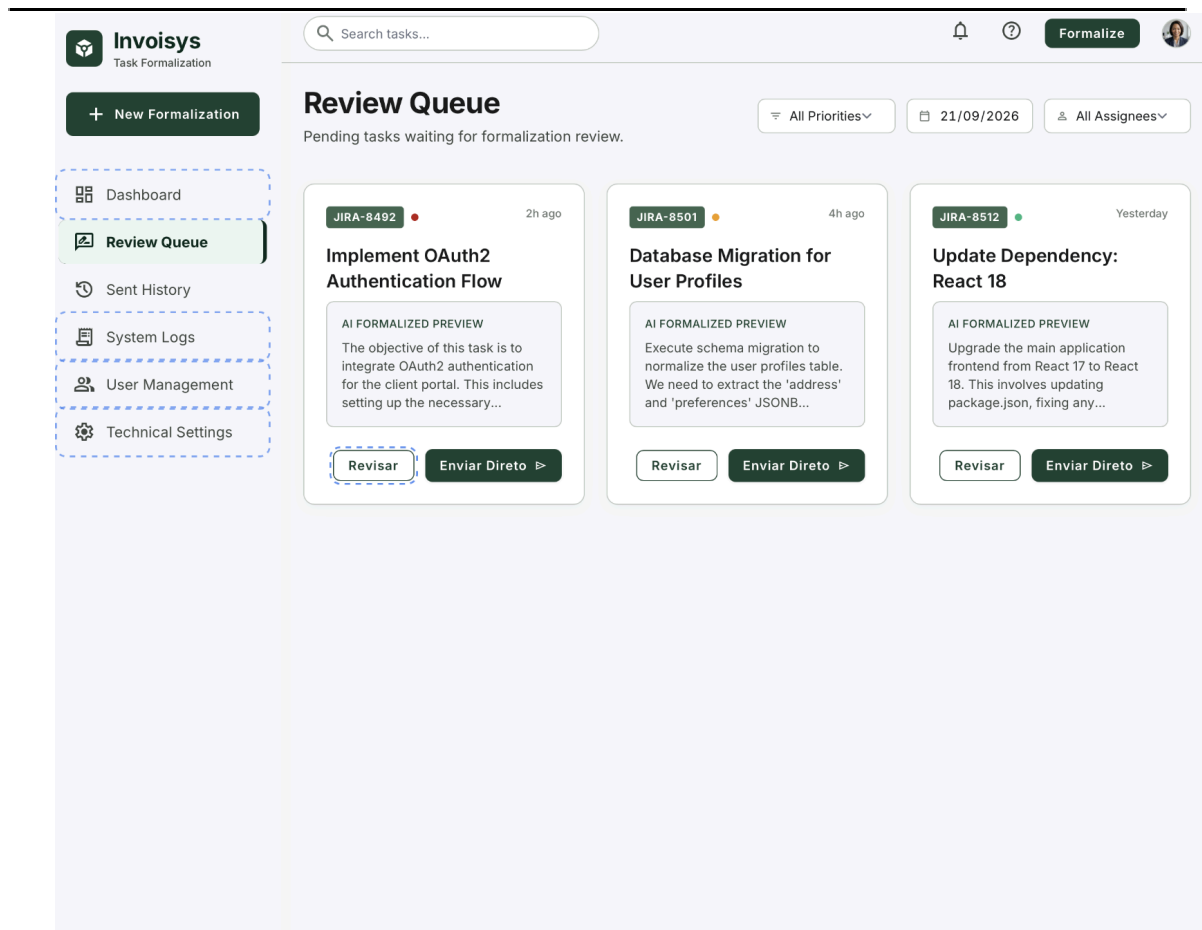


Figura — Fila de Revisão.

Apresenta as tarefas pendentes de revisão e ações para revisar ou enviar o conteúdo.

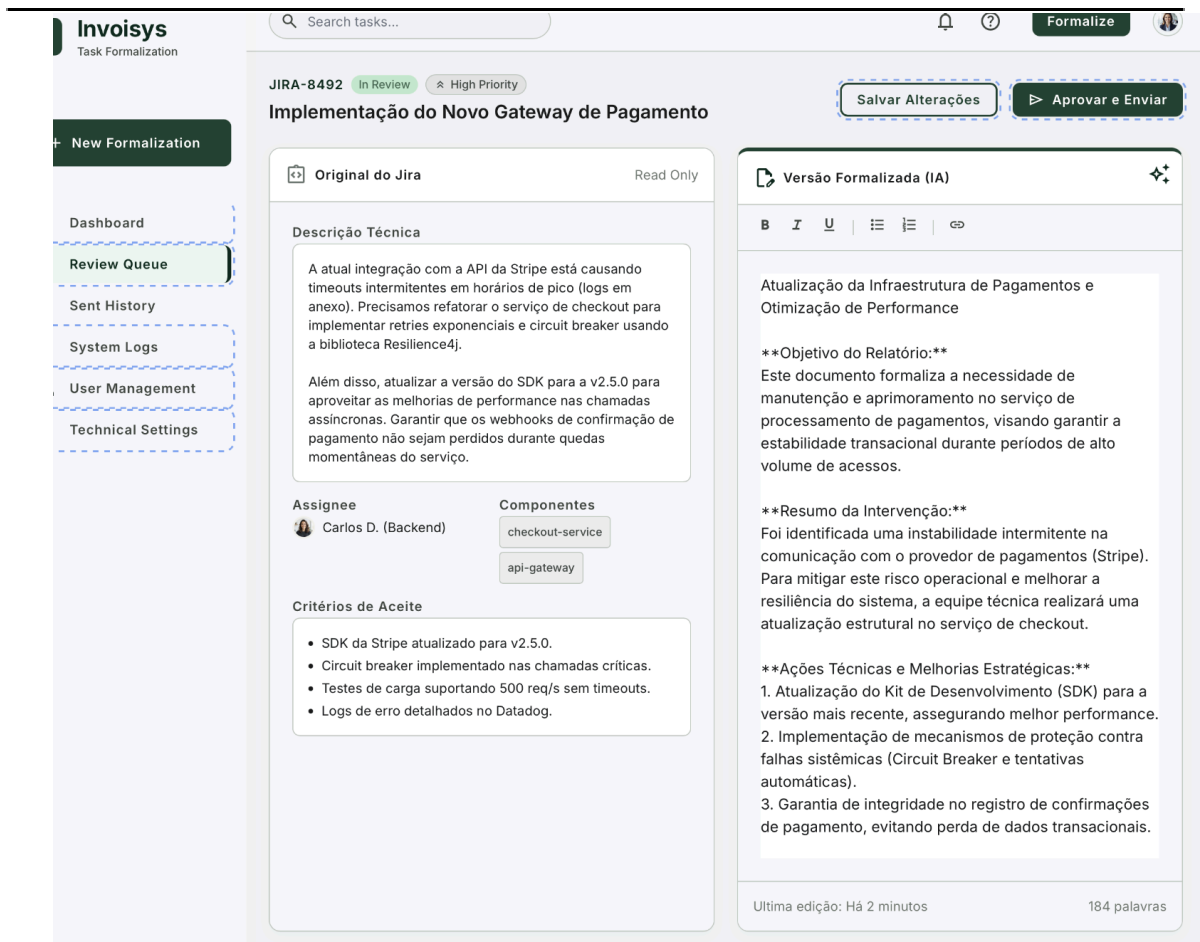


Figura — Revisão de tarefa.

Exibe os dados da história e a versão formalizada pela IA, permitindo revisão, alteração e aprovação.

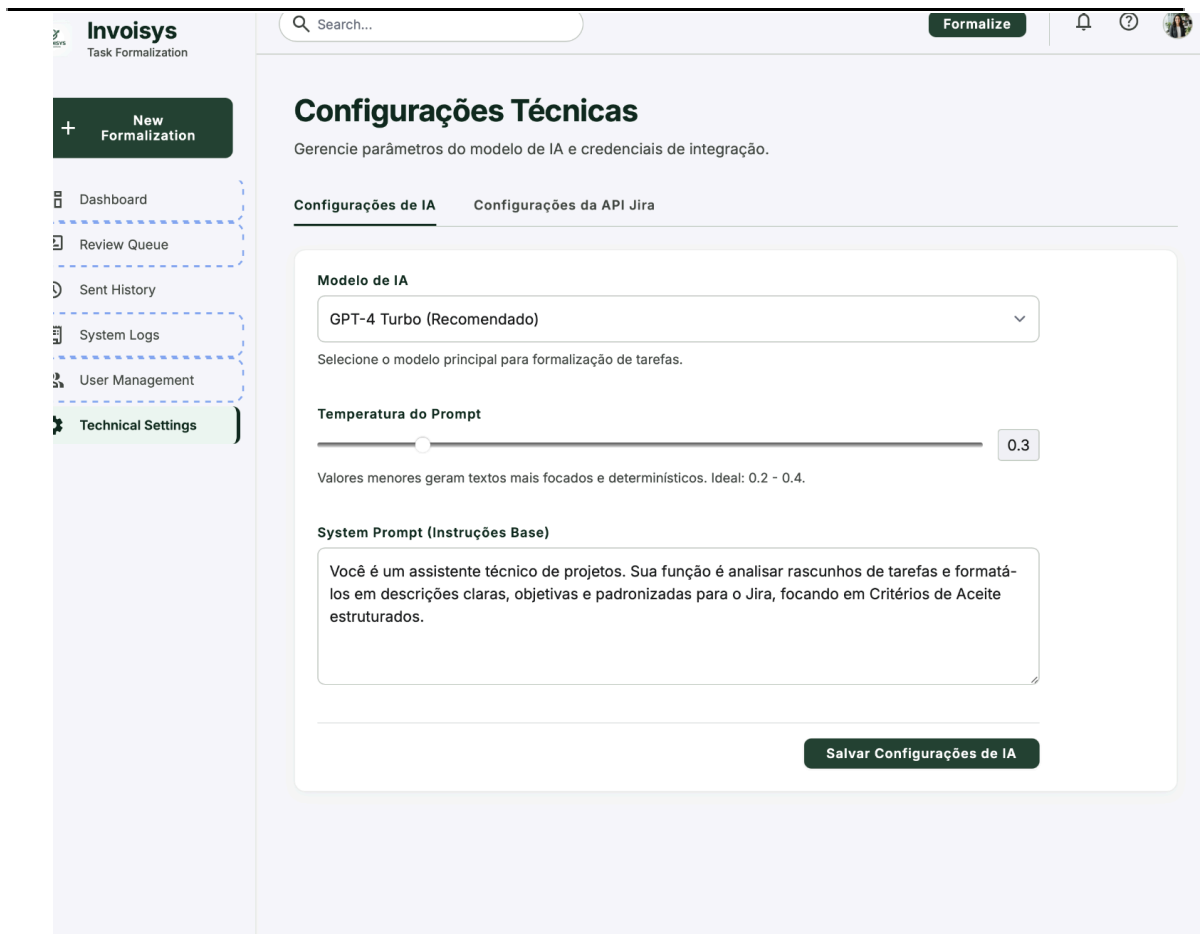


Figura — Configurações de IA.

Permite configurar modelo de IA, temperatura do prompt e instruções do sistema.

Invoisys
Stitch - Design with AI

Search...

Formalize

Dashboard

Review Queue

Sent History

System Logs

User Management

Technical Settings

Novo Usuário

Cadastre um novo colaborador e defina suas permissões de acesso.

Nome Completo
Ex: João da Silva

E-mail Corporativo
joao.silva@empresa.com

Cargo/Função
Selecione um cargo

Permissões Rápidas
Selecione os módulos adicionais que este usuário poderá acessar.

☐ **Acesso a Logs do Sistema**
Visualização do histórico de atividades.

☐ **Configurações de API**
Gerenciamento de chaves e integrações.

☐ **Gestão de Equipes**
Criar e editar grupos de usuários.

☐ **Exportação de Relatórios**
Download de dados em CSV/PDF.

Cancelar Cadastrar Usuário

+ New Formalization

Figura — Criação de usuário.

Permite cadastrar usuário, selecionar cargo e configurar permissões rápidas.

Invoisys
Task Formalization

Formalize

Gestão de Usuários

Gerencie acessos, permissões e histórico da equipe.

Novo Usuário

	Ana Silva ana.silva@invoisys.com	Admin	Ativo
	Carlos Eduardo carlos.ed@invoisys.com	Revisor	Ativo
	Mariana Rios mariana.r@invoisys.com	Visualizador	Inativo

+ New Formalization

Figura — Gestão de usuários.

Apresenta usuários cadastrados, cargos, status e ação para criação de novo usuário.

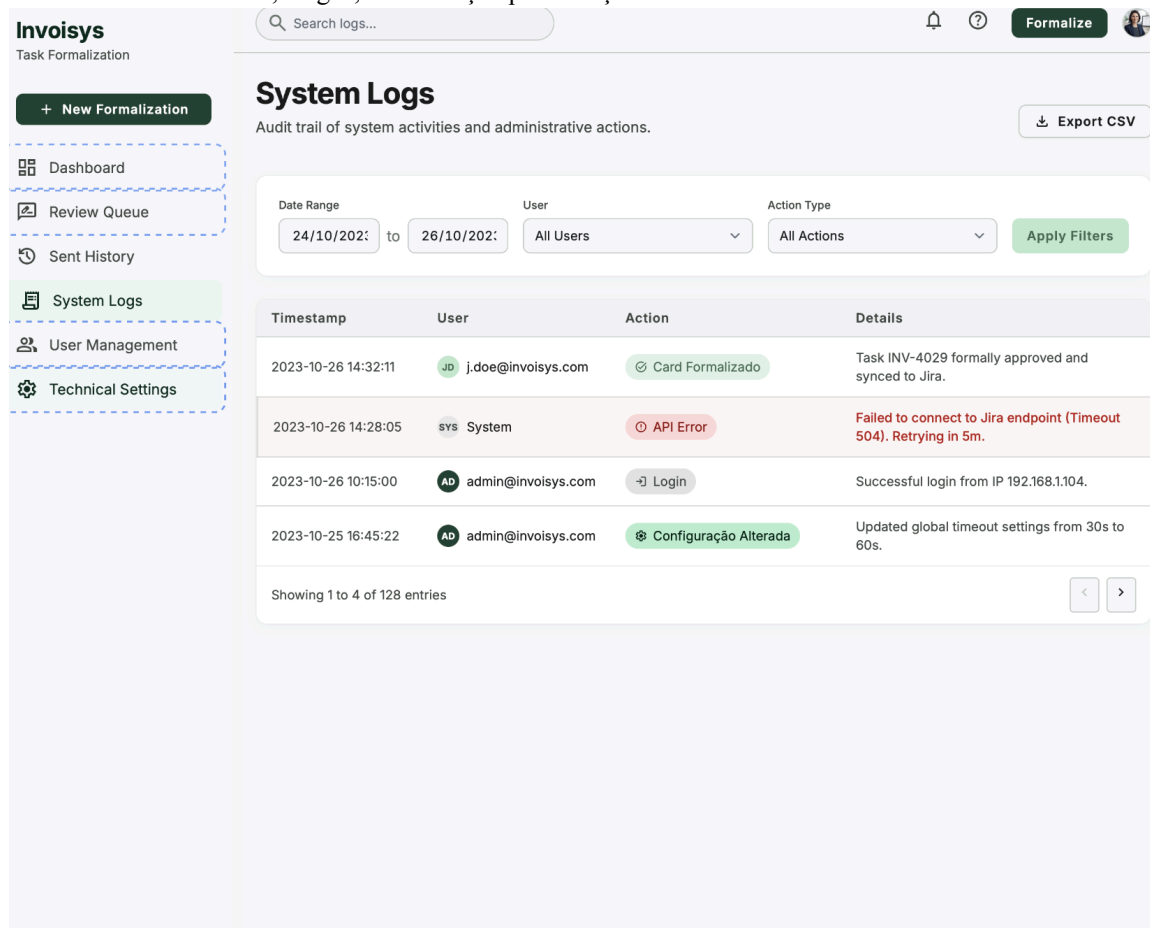


Figura — Logs do sistema.

Apresenta registros de ações administrativas e de processamento, com filtros e possibilidade de exportação.

3.3.1. Diagrama de Navegação

O fluxo de navegação do protótipo parte do Login e segue para recuperação de senha ou confirmação de envio. No desenho funcional futuro, o fluxo de autenticação deverá conduzir ao ambiente de gestão das Releases e revisão dos comunicados. Essa parte do fluxo está representada no Figma, mas ainda não está conectada ao backend atual: o frontend utiliza dados/mock e o código de navegação ainda usa estado local.

3.4. Métricas e Cronograma

Como métrica de escopo documental, foram considerados 16 requisitos funcionais, 11 requisitos não funcionais, 15 regras de negócio, 9 telas prototipadas e 8 entidades/tabelas principais. No código atual, há 2 endpoints de Release implementados, 1 endpoint de health check, 8 entidades de domínio persistentes, 2 adapters de integração externa (Jira e OpenRouter) e uma suíte de testes .NET com 61 casos [Fact] identificados no repositório.

Indicador	Quantidade / referência
Requisitos funcionais	16
Requisitos não funcionais	11
Regras de negócio	15
Telas prototipadas	9
Entidades principais no modelo de dados	8

O cronograma abaixo continua organizado por Sprints, pois os materiais do repositório não fornecem datas oficiais de início e término. A implementação observada no código indica que a base de domínio, modelagem PostgreSQL, integração com Jira, integração com OpenRouter e pipeline principal já possuem implementação, enquanto a integração completa da interface com a API, autenticação e o fluxo REST de revisão/aprovação/exportação ainda precisam ser

concluídos ou confirmados pela equipe.

Etapa	Atividades principais	Resultado esperado
Sprint 1	Levantamento e validação de problema, envolvidos e requisitos.	Escopo e requisitos validados.
Sprint 2	Prototipação e validação das telas no Figma.	Protótipo navegável e requisitos de interface definidos.
Sprint 3	Modelagem do banco, arquitetura e integração inicial com Jira.	Base técnica e persistência estruturadas.
Sprint 4	Implementação do pipeline e integração com IA.	Geração preliminar de Release Note.
Sprint 5	Implementação de revisão, aprovação, histórico e permissões.	Fluxo completo de revisão e aprovação.
Sprint 6	Integração, testes e ajustes finais.	MVP integrado e validado.

4. Análise e Design

Nesta seção, os diagramas foram revisados com base no código efetivamente presente no repositório squad78-invoisys, nas migrations do EF Core, no ERD e na documentação interna de domínio. Quando uma funcionalidade existe apenas no domínio ou no protótipo e ainda não possui endpoint/interface implementados, isso é indicado explicitamente.

4.1. Arquitetura do Sistema

A arquitetura implementada segue uma organização em camadas inspirada em Ports & Adapters. O frontend utiliza React 19 com Vite e Tailwind CSS. O backend utiliza ASP.NET Core 10 com Minimal API. O domínio concentra entidades, enums e portas; a camada Application orquestra o PipelineGeracaoReleaseNote; e a Infrastructure implementa os adapters para Jira, OpenRouter e PostgreSQL por meio do Entity Framework Core 10/Npgsql. Docker Compose disponibiliza a API e o PostgreSQL para execução integrada.

O fluxo de geração atualmente exposto pela API é: o cliente envia a chave da Release; a API chama o pipeline; o pipeline consulta as histórias no Jira, normaliza o texto, agrupa histórias semelhantes, categoriza, reescreve em linguagem de negócio e gera título/resumo por meio do OpenRouter. O resultado é devolvido pela API em DTOs. O domínio também possui regras de revisão, aprovação e reabertura, mas essas operações ainda não estão expostas por endpoints REST no estado atual do repositório.

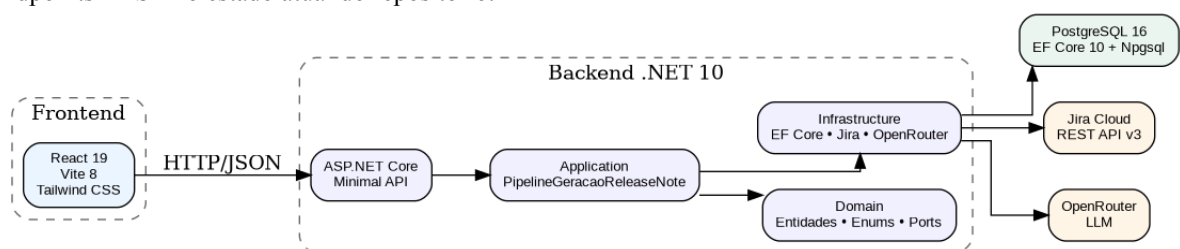


Figura — Arquitetura implementada e integrações externas do Invoisys.

4.2. Modelo do Domínio

O modelo de domínio implementado tem Release como agregado raiz. HistoriaJira representa a entrada bruta do Jira; ExecucaoPipeline registra cada tentativa de processamento; VersaoComunicado representa o comunicado de uma audiência; ItemComunicado representa cada item gerado e revisável; ComunicadoExportado registra uma saída publicada; Usuario representa a identidade de usuário; e DestaqueHero representa conteúdo institucional da tela de login. A separação entre StatusPipeline e StatusRevisao é intencional: a primeira acompanha a geração pela IA e a segunda acompanha a aprovação humana por público.

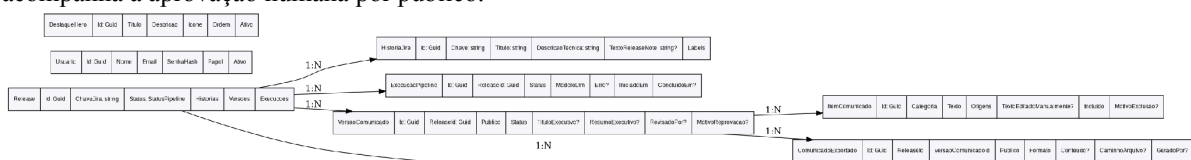


Figura — Modelo de domínio baseado nas entidades efetivamente implementadas.

4.3. Diagramas de Interação

Os diagramas de interação abaixo distinguem o que já está exposto pela API do que existe como regra de domínio. A geração de Release Note possui fluxo REST implementado. Já revisão, reprovação, aprovação e reabertura possuem métodos no domínio, mas ainda dependem da camada API para serem disponibilizados ao frontend.

4.3.1. Diagrama de Sequência

Geração da Release Note

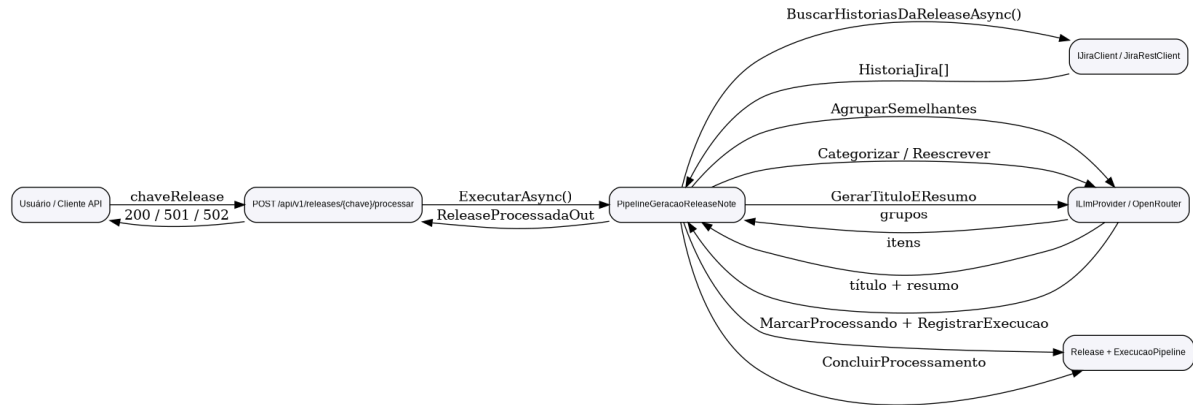


Figura — Sequência implementada para geração da Release Note.

Revisão e Aprovação

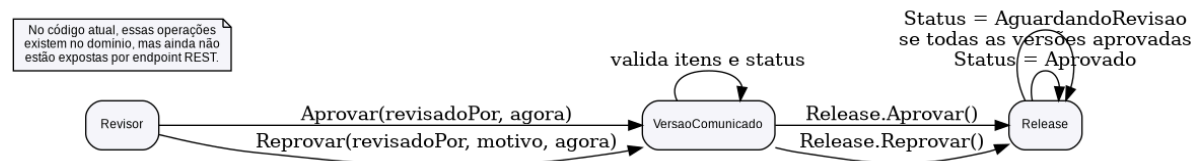
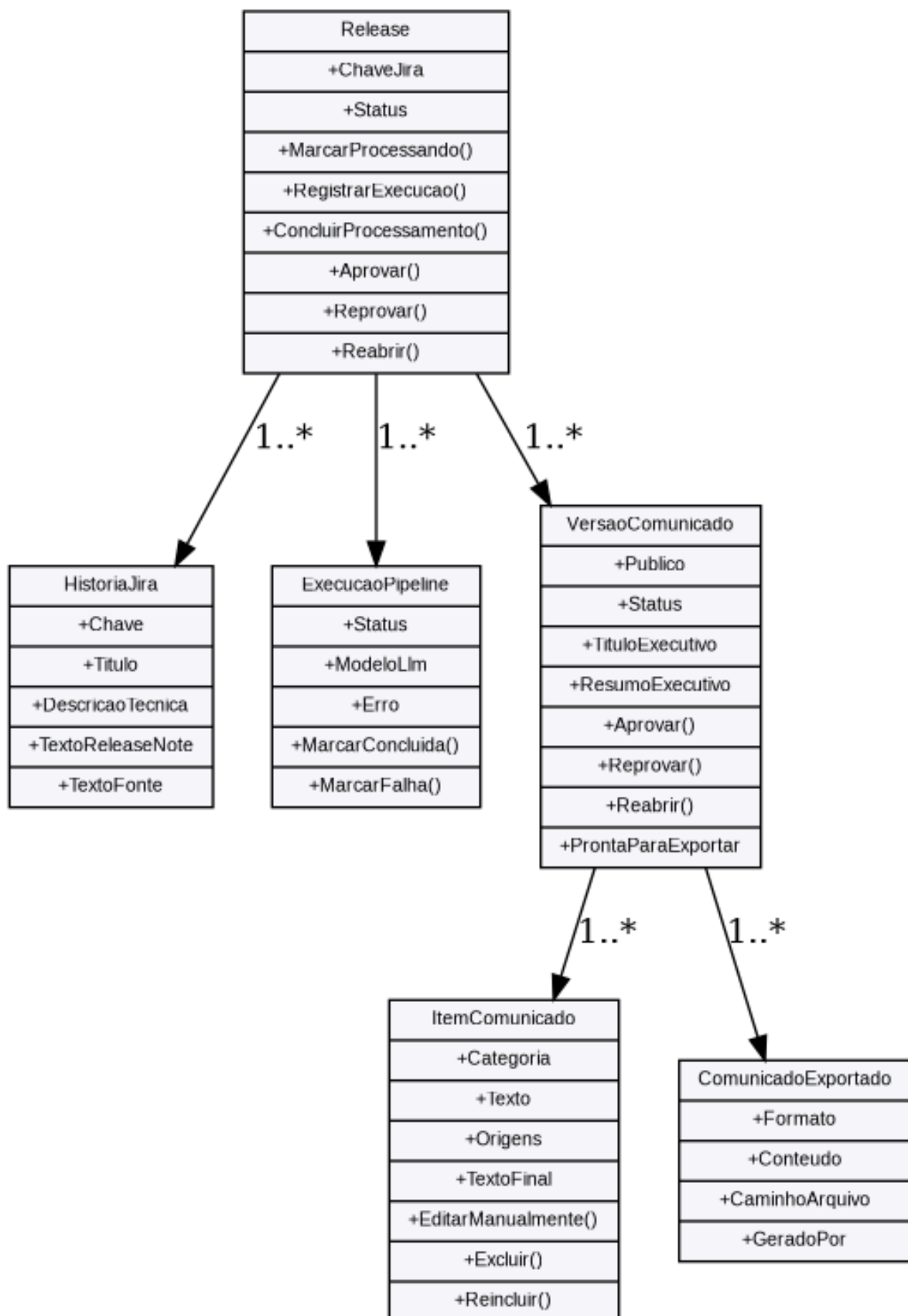


Figura — Sequência das regras de revisão/aprovação existentes no domínio; endpoints REST ainda pendentes.

4.4. Diagrama de Classes

O diagrama de classes foi alinhado às classes efetivamente existentes no projeto. O núcleo é formado por Release, HistoriaJira, ExecucaoPipeline, VersaoComunicado, ItemComunicado e ComunicadoExportado. Usuario e DestaqueHero também são entidades persistentes, porém não participam diretamente do agregado Release. As portas IJiraClient, ILlmProvider e IReleaseRepository representam os pontos de abstração entre domínio/aplicação e infraestrutura.



4.5. Diagrama de Atividades

O processo de geração implementado inicia com a consulta das histórias no Jira e segue pelos cinco estágios definidos no pipeline: extração/limpeza, agrupamento, categorização, reescrita em linguagem de negócio e geração de título/resumo. Ao final, a Release fica aguardando revisão. Falhas de integração ou de resposta do LLM registram a execução como falha e propagam o erro para a API.

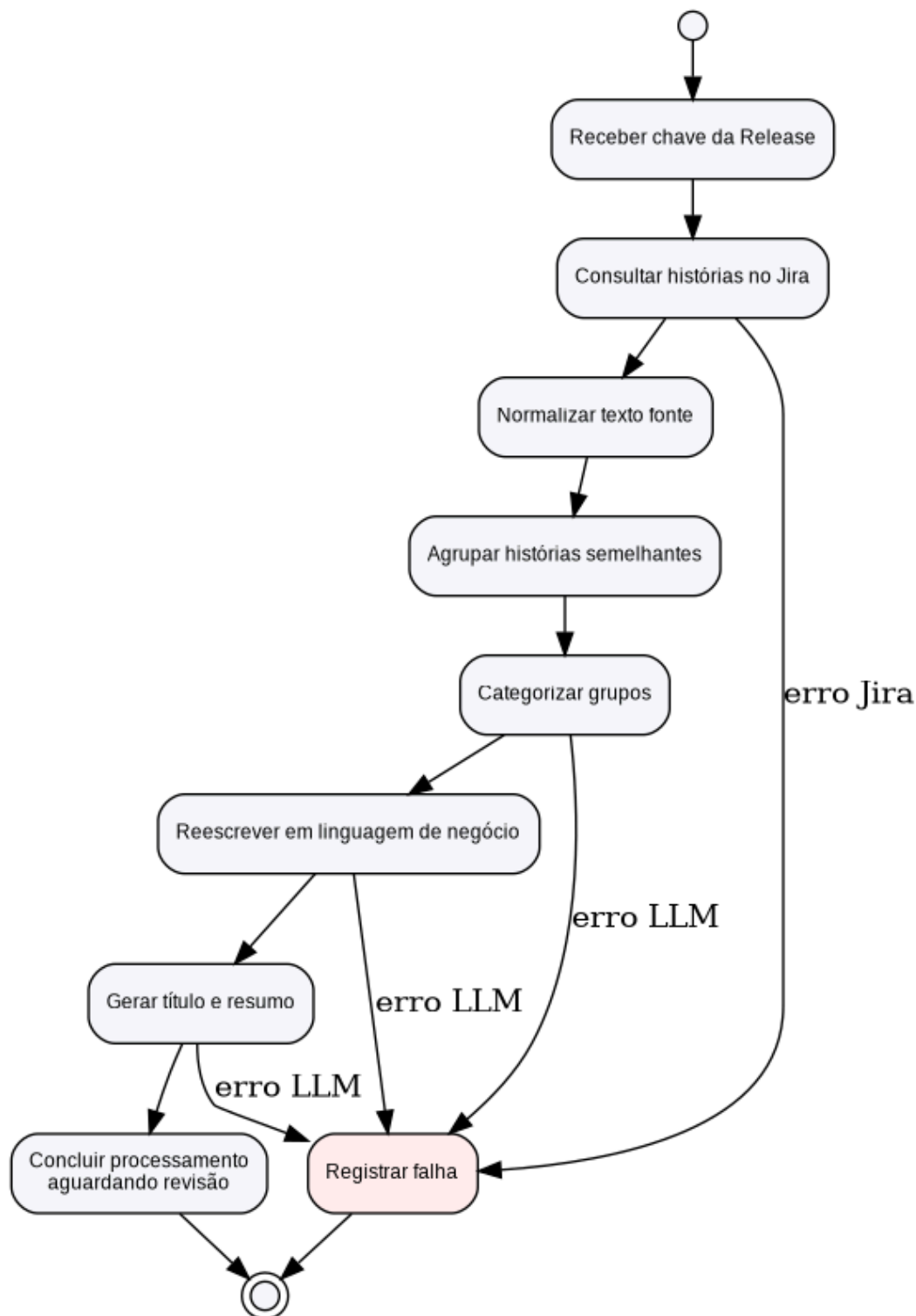


Figura — Atividades do pipeline de geração de Release Note.

Diagrama de Estados

O ciclo de estados implementado deve ser lido em duas dimensões. Release utiliza StatusPipeline: Pendente, Processando, AguardandoRevisao, Aprovado e Falhou. VersaoComunicado utiliza StatusRevisao: AguardandoRevisao, Aprovado e Reprovado. A versão só pode ser exportada quando estiver aprovada e possuir ao menos um item incluído. A separação dos dois estados permite que diferentes públicos tenham ciclos de revisão independentes.

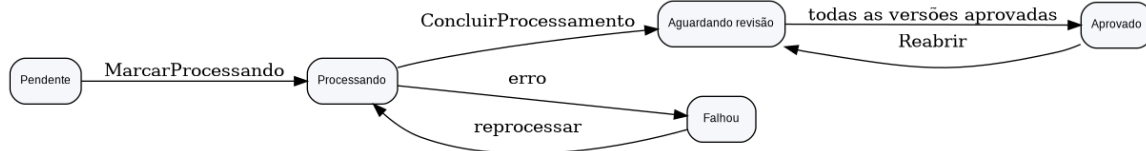


Figura — Estados implementados para a geração da Release e revisão do comunicado.

4.6. Diagrama de Componentes

Os componentes efetivamente identificáveis no repositório são: frontend React/Vite; InvoiSys.Api; InvoiSys.Application; InvoiSys.Domain; InvoiSys.Infrastructure; PostgreSQL; Jira REST API; e OpenRouter. Docker/Docker Compose fornece o ambiente de execução. Componentes como autenticação completa, exportação e endpoints de revisão aparecem no domínio ou no protótipo, mas ainda não constituem componentes REST completos no código atual.

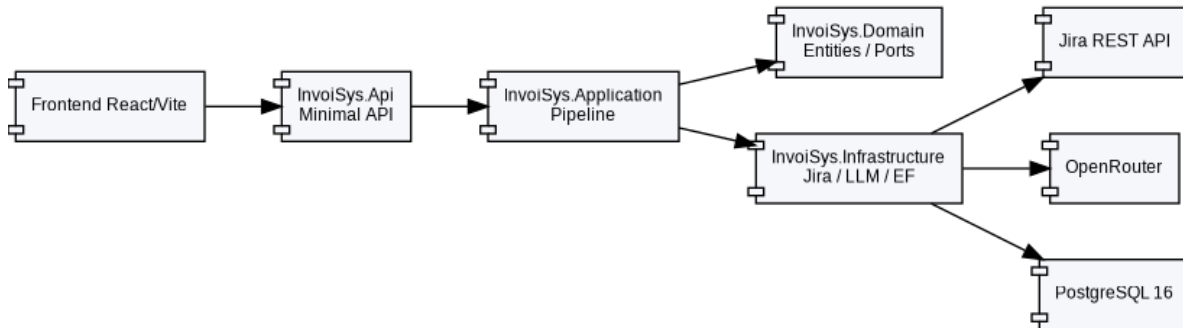


Figura — Componentes e dependências identificados no repositório.

4.7. Modelo de Dados

O modelo de dados efetivamente aplicado pelo projeto utiliza PostgreSQL 16, EF Core 10 Code-First e Npgsql. O DbContext registra oito DbSet: Releases, HistoriasJira, VersoesComunicado, ItensComunicado, Usuarios, ExecucoesPipeline, ComunicadosExportados e DestaquesHero. As migrations InitialCreate e TriggersAtualizadoEm estão versionadas no repositório e representam a criação física do schema.

4.7.1. Modelo Lógico da Base de Dados

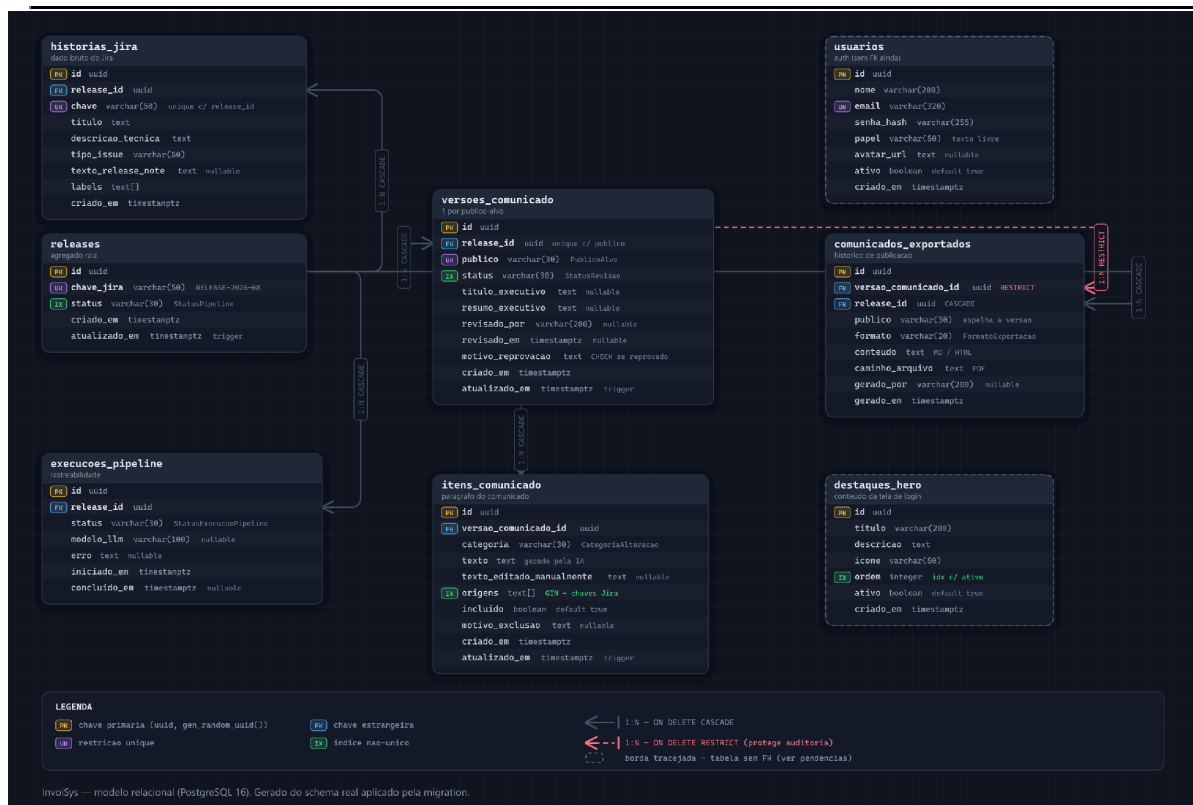


Figura — ERD gerado a partir do schema/migrations reais do PostgreSQL 16.

O relacionamento principal é Release 1:N HistoriaJira, Release 1:N ExecucaoPipeline e Release 1:N VersaoComunicado. Cada VersaoComunicado possui 1:N ItemComunicado e pode possuir 1:N ComunicadoExportado. ComunicadoExportado também referencia Release diretamente. A FK entre ComunicadoExportado e VersaoComunicado utiliza RESTRICT para preservar auditoria, enquanto as demais relações do agregado utilizam CASCADE conforme definido nas migrations.

4.7.2. Criação Física do Modelo de Dados

A criação física é realizada por migrations do Entity Framework Core. A migration 20260918131849_InitialCreate cria as oito tabelas, chaves primárias UUID, chaves estrangeiras, índices, constraints CHECK e índices GIN para origens. A migration 20260918150448_TriggersAtualizadoEm acrescenta os triggers responsáveis pela atualização automática dos campos atualizado_em. O Docker Compose utiliza PostgreSQL 16 e pode aplicar as migrations automaticamente quando Database__MigrarAoIniciar=true.

Diferentemente da versão anterior desta documentação, o repositório enviado contém o script de migrations e o DbContext necessários para reproduzir o modelo físico. Portanto, a estrutura descrita nesta seção pode ser conferida diretamente em src/InvoSys.Infrastructure/Database/Migrations e nas configurações de entidade.

4.7.3. Dicionário de Dados

Tabela releases

Campo	Tipo	Restrição	Descrição
id	uuid	PK	Identificador da Release
chave_jira	varchar(50)	UNIQUE	Chave da Release no Jira
status	varchar(30)	-	Status da Release
criado_em	timestampz	-	Data de criação
atualizado_em	timestampz	trigger	Data de atualização

Tabela historias_jira

Campo	Tipo	Restrição	Descrição
id	uuid	PK	Identificador da história

release_id	uuid	FK	Release relacionada
chave	varchar(50)	UNIQUE/Release	Chave da issue no Jira
titulo	text	-	Título da história
descricao_tecnica	text	-	Descrição técnica
tipo_issue	varchar(50)	-	Tipo da issue
texto_release_note	text	nullable	Texto específico de Release Note
labels	text[]	-	Labels da issue
criado_em	timestampz	-	Data de criação

Tabela execucoes pipeline

Campo	Tipo	Restrição	Descrição
id	uuid	PK	Identificador da execução
release_id	uuid	FK	Release processada
status	varchar(30)	-	Status da execução
modelo_llm	varchar(100)	nullable	Modelo de IA utilizado
erro	text	nullable	Mensagem de erro
iniciado_em	timestampz	-	Início do processamento
concluido_em	timestampz	nullable	Fim do processamento

Tabela versoes comunicado

Campo	Tipo	Restrição	Descrição
id	uuid	PK	Identificador da versão
release_id	uuid	FK/UNIQUE	Release relacionada
publico	varchar(30)	-	Público do comunicado
status	varchar(30)	-	Status da revisão
titulo_executivo	text	nullable	Título do comunicado
resumo_executivo	text	nullable	Resumo executivo
revisado_por	varchar(200)	nullable	Usuário que revisou
revisado_em	timestampz	nullable	Data da revisão
motivo_reprovacao	text	nullable	Motivo quando reprovado
criado_em	timestampz	-	Data de criação
atualizado_em	timestampz	trigger	Data de atualização

Tabela itens comunicado

Campo	Tipo	Restrição	Descrição
id	uuid	PK	Identificador do item
versao_comunicado_id	uuid	FK	Versão relacionada
categoria	varchar(30)	-	Categoria da alteração
texto	text	-	Texto gerado pela IA
texto_editado_manualmente	text	nullable	Texto após edição
origens	text[]	-	Chaves Jira de origem
incluido	boolean	default true	Indica se o item foi incluído
motivo_exclusao	text	nullable	Motivo da exclusão
criado_em	timestampz	-	Data de criação

atualizado_em	timestampz	trigger	Data de atualização
---------------	------------	---------	---------------------

Tabela comunicados exportados

Campo	Tipo	Restrição	Descrição
id	uuid	PK	Identificador da exportação
versao_comunicado_id	uuid	FK RESTRICT	Versão exportada
release_id	uuid	FK CASCADE	Release relacionada
publico	varchar(30)	-	Público da exportação
formato	varchar(20)	CHECK	Formato de saída: markdown, html ou pdf
conteudo	text	-	Conteúdo em Markdown/HTML
caminho_arquivo	text	nullable	Caminho do arquivo PDF
gerado_por	varchar(200)	nullable	Usuário responsável
gerado_em	timestampz	-	Data da exportação

Tabela usuarios

Campo	Tipo	Restrição	Descrição
id	uuid	PK	Identificador do usuário
nome	varchar(200)	-	Nome completo
email	varchar(320)	UNIQUE	E-mail corporativo
senha_hash	varchar(255)	-	Senha armazenada em hash
papel	varchar(50)	-	Papel/permissão do usuário
avatar_url	text	nullable	URL do avatar
ativo	boolean	default true	Indica se o usuário está ativo
criado_em	timestampz	-	Data de criação

Tabela destaques hero

Campo	Tipo	Restrição	Descrição
id	uuid	PK	Identificador do destaque
titulo	varchar(200)	-	Título do destaque
descricao	text	-	Descrição
icone	varchar(50)	-	Identificador do ícone
ordem	integer	index/ativo	Ordem de exibição
ativo	boolean	default true	Indica se está ativo
criado_em	timestampz	-	Data de criação

4.8. Ambiente de Desenvolvimento

O ambiente de desenvolvimento observado no repositório é composto por .NET 10/C# no backend, ASP.NET Core Minimal API, Entity Framework Core 10 com Npgsql, PostgreSQL 16, React 19, Vite 8, Tailwind CSS 3, Docker e Docker Compose. Jira Cloud REST API v3 é utilizado como fonte externa e OpenRouter como gateway de LLM. Git/GitHub são usados para versionamento e colaboração; Figma é usado para prototipação. A suíte de testes utiliza xUnit, FluentAssertions, Microsoft.AspNetCore.Mvc.Testing e Coverlet. O ambiente de produção/hospedagem final não está definido nos arquivos enviados.

Nota de consistência entre documentação e código: o repositório atual apresenta uma diferença importante entre o escopo/protótipo e a implementação. O backend já possui domínio, banco, integração Jira, integração OpenRouter e pipeline de geração, mas a API ainda não expõe autenticação, CRUD de usuários, revisão/aprovação, exportação ou dashboard. Além disso, o frontend atual utiliza mocks e configura endpoints de autenticação/branding que não correspondem aos endpoints expostos pelo backend atual. Essa integração deve ser tratada como pendência de implementação ou alinhamento antes de considerar o fluxo completo do MVP concluído.

Categoria	Tecnologia / ferramenta	Finalidade
Back-end	C# / .NET 10 + ASP.NET Core Minimal API	API REST e exposição dos casos de uso.
Aplicação	InvoiSys.Application	Orquestração do pipeline de geração.
Domínio	InvoiSys.Domain	Entidades, enums, regras e portas.
Infraestrutura	InvoiSys.Infrastructure	Jira, OpenRouter, EF Core e PostgreSQL.
Front-end	React 19 + Vite 8 + Tailwind CSS 3	Interface web e protótipos implementáveis.
Banco de dados	PostgreSQL 16 + EF Core 10 + Npgsql	Persistência relacional e migrations Code-First.
IA	OpenRouter + modelo configurável	Agrupamento, categorização, reescrita e título/resumo.
Integração externa	Jira Cloud REST API v3	Consulta das histórias associadas à Release.
Testes	xUnit + FluentAssertions + ASP.NET Core Testing + Coverlet	Testes unitários e de integração.
Infraestrutura local	Docker + Docker Compose	Execução integrada da API e PostgreSQL.
Versionamento	Git / GitHub	Branches, commits, revisão e colaboração.
Prototipação	Figma	Modelagem e validação visual das telas.

A definição detalhada de hardware, rede, hospedagem e infraestrutura de produção não consta nos materiais fornecidos nesta etapa. Esses itens deverão ser especificados quando o ambiente de implantação for definido.